

Rapport TER

Zinck Tom - Saperès Clément - Nigh Kai - Bonetti Timothée

may 2025

Table des matières

1	Introduction	2
2	Planification	4
2.1	Partage des tâches	4
2.2	Design du gameplay	5
2.2.1	Le rôle du Seigneur	5
2.2.2	Le rôle du Péon	7
3	État de l’art	8
3.1	A Review of Digital Terrain Modeling [7]	8
3.2	AutoBiomes : procedural generation of multi-biome landscapes (2020) [5]	12
3.3	papier 2 UI [12]	14
3.4	papier 4	15
4	développement	16
4.1	Génération de la carte	17
4.2	Moteur de jeu	20
4.2.1	Technologies	20
4.2.2	Le design d’un moteur de jeu	20
4.2.3	Entity-component-system	21
4.2.4	Notre moteur	22
4.2.5	ECS	23
4.2.6	Components (<i>Core</i>)	23
4.2.7	Systems (<i>Core</i>)	24
4.2.8	Input	24
4.2.9	User Interface	25
4.2.10	Game (Classe)	26
4.3	Implémentation de la caméra	27
4.4	Assets	29
4.5	communication avec le backend	30
5	Conclusion	30

1 Introduction

L'objectif de ce projet est de réaliser un jeu vidéo massivement multijoueur jouable directement depuis un navigateur web. Ce jeu propose deux modes de jeu bien distincts, offrant aux joueurs des expériences complémentaires :

- *Mode "Péon"* : le joueur incarne un personnage individuel et s'occupe d'actions élémentaires comme la collecte de ressources, la construction de bâtiments, ou d'autres tâches concrètes. Ces actions sont réalisées à travers des mini-jeux simples et interactifs.
- *Mode "Seigneur"* : le joueur ne contrôle pas directement un personnage, mais adopte une vue stratégique à l'échelle d'un royaume. Il donne des ordres aux péons en fonction des priorités et des besoins du territoire qu'il gouverne.

L'univers de jeu est composé de plusieurs royaumes, chacun dirigé par un seigneur différent. Ces royaumes sont en concurrence pour devenir la puissance dominante. Ce système s'inspire librement de la célèbre "Pixel War", un événement sur la plateforme Reddit (r/Place)[10], où des milliers d'utilisateurs ont collaboré ou se sont affrontés pour s'approprier des espaces d'un immense canevas numérique. Bien que purement esthétique, cet événement a généré de véritables dynamiques sociales, entre alliances communautaires et rivalités de territoires. Notre ambition est de transposer ces dynamiques sociales dans un jeu structuré et compétitif, en donnant du sens aux interactions et aux conflits entre joueurs.

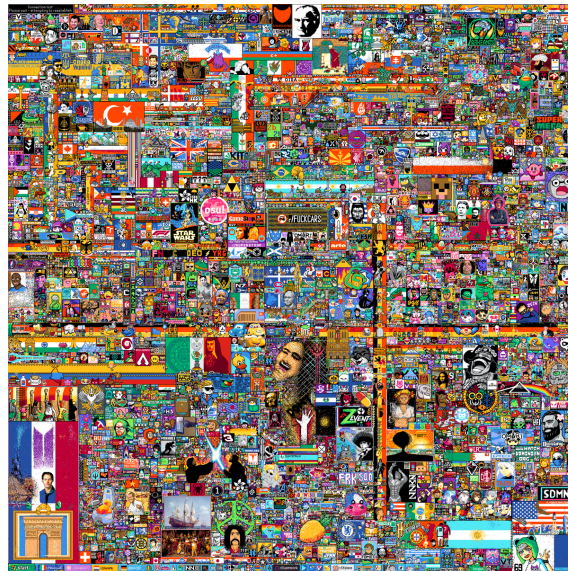


FIGURE 1 – Image finale de r/Place

Étapes et collaboration Ce projet, bien que complet en soi, est réalisé en collaboration avec un autre groupe de TER ayant développé un backend scalable destiné à supporter des applications en ligne à fort trafic. Cette coopération nous permet de nous concentrer sur le développement du frontend, tout en assurant une infrastructure serveur solide et adaptée à un jeu massivement multijoueur.

Défis identifiés Avant même de commencer le développement, nous avons identifié plusieurs défis techniques et conceptuels majeurs :

- Générer une carte procédurale intégrant différentes ressources.
- Choisir une technologie adaptée au développement web interactif (WebGL, WebSockets, etc.).
- Concevoir un moteur de jeu simple et modulaire.
- Développer une interface utilisateur intuitive et réactive.

- Créer des mini-jeux simples mais engageants pour les actions de type Péon.
- Équilibrer les deux modes de jeu (Péon / Seigneur) pour garantir une jouabilité équitable et cohérente.
- Assurer une communication fluide et sécurisée entre frontend et backend.
- Créer ou intégrer des assets graphiques adaptés à l'univers du jeu.

Portée du projet (MVP) Ce projet est ambitieux, et il serait irréaliste de livrer une version complète, stable et publiable dans le temps imparti. C'est pourquoi notre objectif est de développer un *prototype fonctionnel* (MVP – Minimum Viable Product). Ce MVP se concentrera sur l'essentiel : la mise en œuvre des deux modes de jeu, l'interaction avec une carte interactive, et la connexion avec le backend pour la gestion en temps réel des actions des joueurs. Il servira de base pour valider les choix techniques, les mécaniques de gameplay, et les dynamiques collectives que nous souhaitons créer. Ce projet sera continué à la suite de ce rendu dans le but de réelement le lancer en ligne.

Problématique *Comment concevoir un prototype fonctionnel de jeu massivement multijoueur sur navigateur qui permette de valider les dynamiques de coopération et de stratégie entre joueurs, tout en intégrant de manière cohérente et interactive deux modes de jeu complémentaires (Péon et Seigneur) ?*

2 Planification

2.1 Partage des tâches

La planification de notre projet a été structurée en quatre grandes phases : la recherche et la conception, la création du jeu, le développement des fonctionnalités en ligne, et la phase de finalisation. Le diagramme de Gantt ci-dessous illustre cette répartition temporelle et thématique.

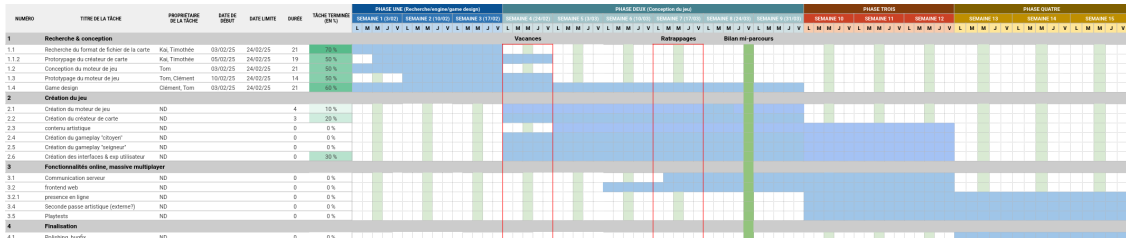


FIGURE 2 – Diagramme de Gantt

Dès le début, nous avons identifié trois objectifs fondamentaux : développer un outil de génération de carte en C++, établir les bases du moteur de jeu en TypeScript (notamment à l'aide de la bibliothèque Three.js), et définir le design général du jeu. En conséquence, les tâches ont été réparties selon les affinités et compétences de chacun : Tom Zink a pris en charge le développement du moteur de jeu, Clément Saperes s'est concentré sur le game design, tandis que Kai Night et Timothée Bonetti ont collaboré sur l'outil de création de carte.

Les premières semaines ont été majoritairement dédiées à cette phase de recherche et de prototypage. Une fois ces bases posées, la deuxième phase du projet consistait à réunir les différents composants pour créer un gameplay cohérent (incluant les rôles « citoyen » et « seigneur »), concevoir les interfaces utilisateurs et assurer une interactivité fluide.

Imprévu Cependant, cette intégration s'est révélée plus complexe que prévu. Plusieurs obstacles ont freiné notre progression : une méconnaissance initiale de certaines technologies clés (notamment TypeScript et Three.js), le manque de bibliothèques satisfaisantes pour la création d'interfaces web, ainsi que des difficultés à coordonner notre travail en raison d'emplois du temps peu compatibles. Ces contraintes ont nécessité une réévaluation continue de nos priorités et ont impacté l'avancement de certaines tâches, comme en témoigne le taux d'achèvement partiel de certaines activités dans le diagramme. Nous reviendrons plus en détail sur ces difficultés dans les sections suivantes de ce rapport.

2.2 Design du gameplay

Dans cette section, nous présentons les idées de gameplay ainsi que les mécaniques de jeu que nous souhaitons implémenter dans la version finale.

2.2.1 Le rôle du Seigneur

Les joueurs qui choisissent d'incarner un Seigneur ont pour objectif de développer le royaume le plus vaste, le plus esthétique et le plus prospère possible. Pour ce faire, ils interagissent principalement avec une série d'interfaces leur permettant de construire ou d'améliorer des bâtiments, de gérer la diplomatie, ou encore de superviser les échanges commerciaux avec les royaumes voisins.

Ces joueurs passent donc une grande partie de leur temps dans ces interfaces. Il était essentiel d'apporter un soin particulier à leur conception. C'est notamment pour cette raison que nous avons décidé d'implémenter notre propre classe d'interface. Ce choix sera détaillé dans une section ultérieure. Par ailleurs, nous avons conçu une maquette fonctionnelle détaillée des interfaces du Seigneur à l'aide de l'outil *Figma* [11], en nous concentrant sur l'organisation fonctionnelle plutôt que sur l'aspect visuel.

À travers ces interfaces, le Seigneur choisit les bâtiments à construire ou à améliorer. Ce sont ensuite les Péons qui collectent les ressources nécessaires et bâtissent les édifices. Le Seigneur peut également conclure des accords commerciaux avec d'autres Seigneurs, ou établir des pactes diplomatiques. Enfin, seul lui a accès à l'ensemble des informations détaillées sur les ressources du royaume, ce qui incite les autres joueurs à coopérer et à dialoguer.

Le Seigneur dispose d'un contrôle libre de la caméra, aussi bien en vue globale qu'en vue détaillée. En vue globale, il peut visualiser les niveaux de ressources de son royaume ainsi que des informations générales sur ses villes.

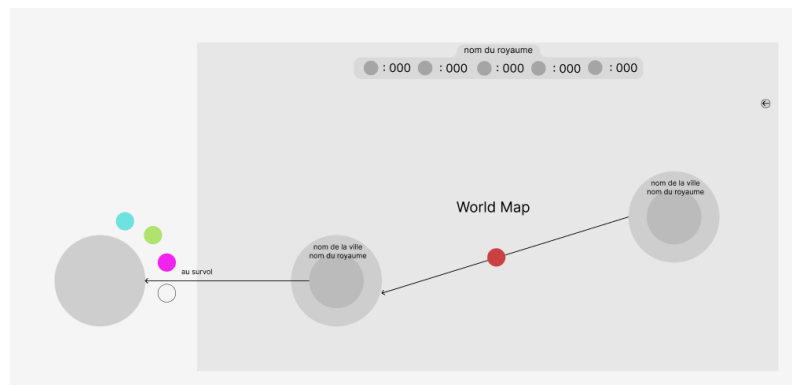


FIGURE 3 – Schéma de l'interface en vue globale

En cliquant sur une ville, le Seigneur accède à une interface spécifique lui permettant de consulter diverses informations telles que les bâtiments construits, ceux en cours de construction, et ceux dont il peut ordonner l'édification par les Péons.

Un autre aspect important du rôle du Seigneur est la gestion diplomatique avec les autres royaumes. Il peut discuter en privé avec les autres Seigneurs, proposer des accords commerciaux, des alliances, ou encore déclarer des guerres.

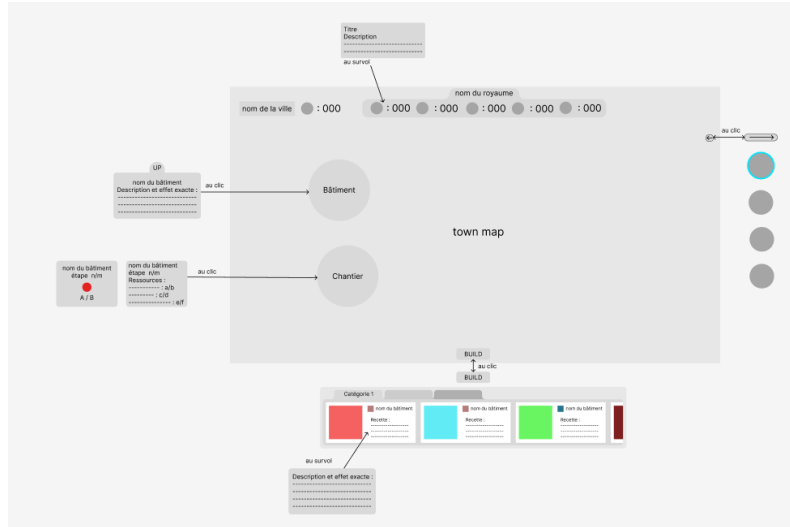


FIGURE 4 – Schéma de l'interface de gestion de ville

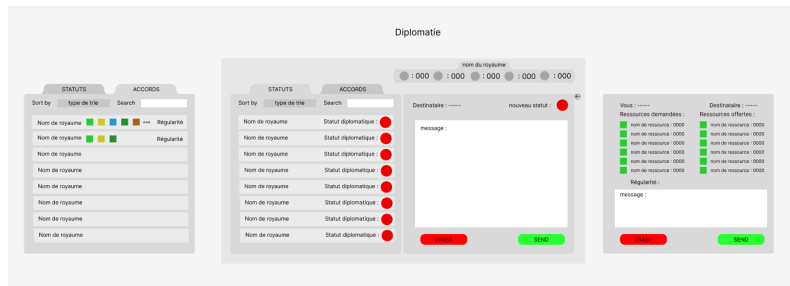


FIGURE 5 – Schéma de l'interface diplomatique

Modélisation simplifiée des bâtiments Une contrainte importante réside dans la limitation des échanges d'informations entre le serveur et les clients, afin d'éviter une surcharge du réseau. Pour répondre à cette contrainte, nous avons choisi un système de représentation des bâtiments à la fois simple et efficace :

- Chaque bâtiment possède un **type**, défini par son niveau, ainsi qu'un **état de santé** correspondant à son avancement dans le processus de construction.
- Lorsqu'un Seigneur place un nouveau bâtiment ou améliore un bâtiment existant, celui-ci commence avec **0 point de vie** et un **dictionnaire de ressources** listant les matériaux nécessaires pour l'étape de construction en cours.
- Les Péons apportent les ressources requises, complétant progressivement le dictionnaire.
- Une fois le dictionnaire complété, ils peuvent effectuer des *mini-jeux de construction* pour augmenter les points de vie du bâtiment.
- Lorsqu'un seuil donné de points de vie est atteint, un nouveau dictionnaire est généré pour l'étape suivante, et le processus recommence.

Chaque bâtiment peut ainsi comporter plusieurs étapes de construction, chacune nécessitant un nombre variable de mini-jeux. Ce système permet de limiter les échanges réseau aux informations essentielles (état de santé et ressources), tandis que les détails des étapes sont stockés localement.

2.2.2 Le rôle du Péon

Le gameplay du Péon est beaucoup plus simple. Son rôle consiste à exécuter les tâches élémentaires : il peut se déplacer librement sur la carte et interagir avec son environnement. Par exemple, interagir avec une ressource déclenche un mini-jeu qui détermine s'il parvient ou non à l'obtenir. Il peut ensuite transporter cette ressource, par exemple vers un chantier de construction. Ce sont également les Péons qui construisent les bâtiments, via d'autres mini-jeux.

Mini-jeux Les Péons sont responsables de la collecte des ressources et de la construction des bâtiments. Pour rendre ces actions plus engageantes, nous avons décidé d'intégrer des mini-jeux simples à la place d'actions automatiques (comme cliquer sur un arbre pour obtenir du bois). Ces mini-jeux prennent la forme de nouvelles scènes `Three.js` qui s'exécutent en dehors du système ECS principal (Qui sera détaillé dans la section concernant le moteur de jeu). Lorsqu'un joueur interagit avec une ressource ou un objet, une fenêtre de mini-jeu s'affiche au centre de son écran. Le type de mini-jeu dépend de l'action à effectuer, et plus le joueur réussit le mini-jeu, plus il obtient de ressources.

Par exemple, dans le mini-jeu de coupe du bois, le joueur doit effectuer des allers-retours avec sa souris, simulant une scie. Plus les mouvements sont efficaces, plus il récolte de bûches.

3 État de l'art

3.1 A Review of Digital Terrain Modeling [7]

Ce papier de recherche fait un tour d'horizon des méthodes de génération de terrain par ordinateur. Le papier étant fourni nous nous attarderons uniquement sur les parties qui nous semblent les plus pertinentes.

Représentation du terrain La modélisation numérique d'un terrain repose sur une fonction d'élévation $h : \mathbb{R}^2 \rightarrow \mathbb{R}$ de classe au moins continue (C^0), qui associe une altitude à chaque point du plan. Cette définition suppose l'absence de surplombs, d'arcs ou de cavernes. Le terrain T est défini sur un domaine $\Omega \subset \mathbb{R}^2$, souvent un rectangle $R(a, b)$ dont a et b sont des coins opposés. La pente en un point p est donnée par la norme du gradient $\|\nabla h(p)\|$, tandis que la normale au terrain s'exprime par la projection $(-\nabla h(p), 1)$ dans $\mathbb{R}^3$, puis normalisée.

L'élévation peut être représentée par une fonction explicite (analytique ou procédurale). Bien que cette approche soit compacte en mémoire et offre une précision théorique infinie, elle peut être coûteuse en calcul. En pratique, la représentation la plus courante est la grille régulière de valeurs d'altitude, appelée *champ de hauteurs* (ou *Digital Elevation Model*, DEM). Elle consiste en une grille 2D régulière sur laquelle les altitudes z_{ij} sont échantillonnées aux points p_{ij} selon une subdivision en n intervalles. Cette méthode nécessite de reconstruire la surface terrain par interpolation (bilinéaire ou bicubique). L'interpolation bilinéaire assure une continuité C^1 à l'intérieur des cellules, mais seulement C^0 à leurs bords. Les interpolations d'ordre supérieur (par exemple bicubique) sont plus précises mais plus coûteuses en calcul.

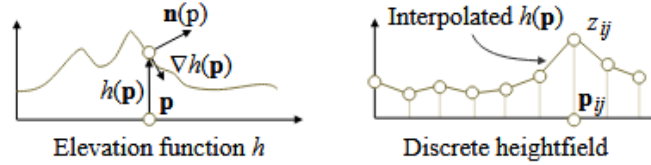


FIGURE 6 – Difference entre représentation explicite et champ de hauteur

Bien que les champs de hauteurs soient plus gourmands en mémoire ($O(n^2)$), ils permettent de représenter divers reliefs, contrairement aux fonctions explicites. Des techniques comme la quantification (souvent en 8 ou 16 bits) sont utilisées pour réduire les besoins mémoire. Cependant, une précision de 8 bits est insuffisante pour des applications telles que la génération procédurale ou la simulation, car elle provoque des artefacts visuels. Les champs de hauteurs sont largement utilisés dans les outils de création de terrain, les simulations d'érosion, les jeux vidéo et l'industrie cinématographique (souvent combinés à des structures 3D plus complexes).

Génération procédurale de terrains Dans cette étude, on qualifie de *procédurale* toute méthode de génération de terrain qui ne repose ni sur une simulation physique réelle (comme l'érosion hydraulique), ni sur l'utilisation de données issues du monde réel. Ces approches cherchent plutôt à reproduire directement les effets observés dans la nature, en s'appuyant sur des propriétés générales des terrains comme leur caractère fractal. Elles s'inspirent des formes naturelles, mais sans utiliser de données réelles comme exemples.

Les méthodes procédurales se divisent en deux grandes catégories. D'une part, la génération de terrains à grande échelle vise à synthétiser un relief sur l'ensemble du plan $\mathbb{R}^2$ ou un domaine étendu $\Omega \subset \mathbb{R}^2$. Ces méthodes exploitent les propriétés d'auto-similarité des reliefs à différentes échelles, mais offrent généralement peu de contrôle sur la forme finale. D'autre part, les méthodes de génération de formes de terrain spécifiques (par exemple collines, rivières ou canyons) travaillent à plus petite échelle et permettent un contrôle plus direct ou indirect sur la géométrie générée grâce à des paramètres ajustables.

Génération à grande échelle par subdivision Les terrains naturels tels que les chaînes de montagnes érodées ou les littoraux présentent souvent des structures similaires aux propriétés fractales. De nombreux algorithmes de génération à grande échelle exploitent ces caractéristiques, produisant des détails qui se répètent à toutes les échelles.

Parmi ces algorithmes, les *schémas de subdivision* raffinent itérativement une grille initiale pour ajouter des détails de plus en plus fins. L'un des plus connus est l'algorithme de déplacement de points médians (*midpoint displacement*), qui introduit de nouveaux points en moyenne des voisins existants, puis applique un déplacement aléatoire dont l'amplitude décroît à chaque niveau d'itération, en fonction de la dimension fractale souhaitée.

Pour limiter les artefacts directionnels visibles (par exemple les pics aux premiers niveaux de subdivision), plusieurs variantes ont été proposées. Le schéma *diamond-square* présenté dans la figure 7, bien que populaire, présente des extrema locaux visibles. Le schéma *square-square* améliore cet aspect en équilibrant mieux les subdivisions. D'autres schémas utilisent des combinaisons de polygones (triangles, carrés...) pour conserver la cohérence géométrique et topologique lors du raffinement.

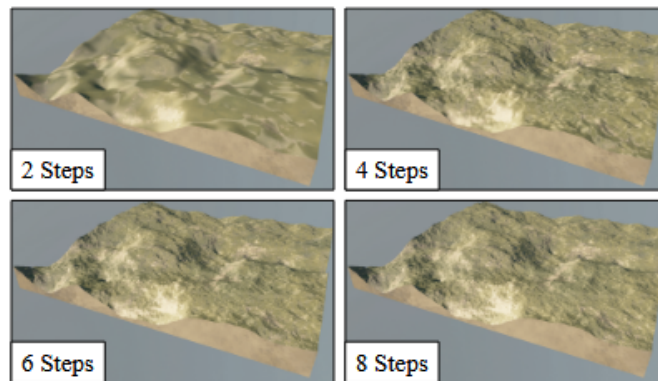


FIGURE 7 – Exemple algorithme diamond square

Génération par fonctions de bruit Les fonctions de bruit sont largement utilisées dans la génération procédurale de terrains. Elles permettent de modéliser des reliefs variés en combinant plusieurs bruits à différentes échelles et amplitudes. L'un des modèles les plus courants est le mouvement brownien fractionnaire (fBm)[8], obtenu en sommant plusieurs octaves d'une fonction de bruit lisse avec des fréquences et amplitudes variables. Ces paramètres, appelés *lacunarity* et *persistence*, influencent respectivement la variation de fréquence et d'amplitude d'une octave à l'autre, et donc l'aspect global du terrain : plus rugueux ou plus doux.

Pour simuler des crêtes ou des arêtes plus marquées, des variantes comme le *ridge noise* (ou bruit de crête) inversent la forme du bruit pour créer des sommets nets, adaptés aux chaînes de montagnes jeunes.

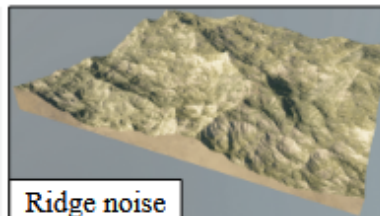


FIGURE 8 – Exemple bruit de crête

D'autres extensions, comme le *bruit multifractal*, modifient dynamiquement l'amplitude des octaves selon l'altitude locale, afin de mieux reproduire les différences entre plaines douces et montagnes accidentées.

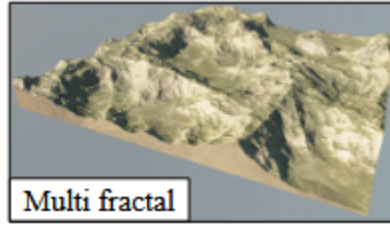


FIGURE 9 – Exemple bruit multifractal

Enfin, pour éviter les artefacts de grille souvent visibles dans les sommes simples de bruits, on applique des transformations appelées *warping* (enroulé) au domaine d'entrée de la fonction. Cela permet de déformer le terrain de manière plus organique.

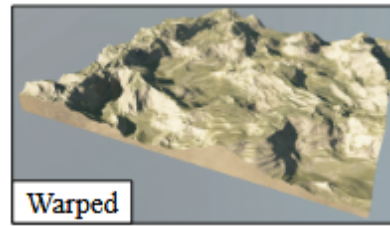


FIGURE 10 – Exemple bruit multifractal avec enroulage

Bien que ces techniques puissent imiter certains effets d'érosion, elles restent des approximations, et des simulations physiques plus complexes sont nécessaires pour des résultats réalistes. La figure 8 montre des exemples d'application de ces méthodes pour la génération de terrain.

Limites des approches fractales pour la génération de terrains Les approches basées sur la subdivision ou les fonctions de bruit sont couramment utilisées pour générer des terrains synthétiques à grande échelle, en raison de leurs fortes propriétés fractales. Toutefois, ces méthodes peinent à reproduire les structures de haut niveau que l'on observe dans les terrains réels, comme les ramifications caractéristiques des chaînes de montagnes. Les fonctions de bruit offrent une grande flexibilité, notamment grâce aux techniques de déformation (*warping*), de lissage ou de combinaison, mais leur contrôle reste complexe et nécessite à la fois une bonne maîtrise technique et un sens artistique développé. À l'inverse, les méthodes par subdivision, bien que plus simples à manipuler, sont moins présentes dans les outils industriels, principalement en raison de leur moindre compatibilité avec le calcul parallèle.

L'un des avantages majeurs des fonctions de bruit réside dans leur capacité à être évaluées indépendamment pour chaque point, ce qui les rend particulièrement efficaces sur GPU pour l'édition interactive. Néanmoins, cette indépendance constitue aussi une limite importante : ces fonctions ne tiennent pas compte des relations locales entre les points, et ne permettent donc pas de modéliser des phénomènes complexes tels que le transport de matière ou l'érosion. En outre, le caractère strictement fractal de ces méthodes ne correspond pas à la diversité morphologique réelle des paysages, ce qui restreint leur réalisme à certaines échelles ou à des formes spécifiques comme les côtes fractales. Ces limites ont conduit la recherche à se tourner vers des modèles plus spécialisés, capables de générer des formations géographiques précises tout en conservant les performances des approches procédurales.

Méthodes de génération basées sur les entités géographiques Les méthodes dites "feature-based" reposent sur la modélisation explicite de structures géographiques clés comme les crêtes, les vallées ou les lits de rivières. Deux grandes approches se distinguent : d'une part, les techniques basées sur des courbes de contrôle permettent à l'utilisateur de dessiner les grandes lignes du relief à partir de profils et de silhouettes, qui guident ensuite la déformation du terrain. Ce type de modélisation, souvent interactif, permet une grande expressivité et un contrôle précis sur la forme

finale du terrain. D'autre part, les méthodes par "arbres de construction" combinent des primitives de terrain (comme collines, montagnes, rivières) hiérarchiquement à l'aide d'opérations de fusion ou de déformation. Ces approches permettent de construire des terrains complexes en associant des éléments topographiques à différentes échelles. En s'appuyant sur des structures géographiques réelles (réseaux hydrographiques, pentes, courbures), ces méthodes offrent un compromis intéressant entre contrôle utilisateur, réalisme géomorphologique et potentiel procédural.

Conclusion L'étude des méthodes présentées par cet article nous a permis de connaître les méthodes qui existent pour générer du terrain par ordinateur, leurs avantages et inconvénients. Nous avons explorés ces pistes pour créer notre carte.

3.2 AutoBiomes : procedural generation of multi-biome landscapes (2020) [5]

Approche proposée pour la génération procédurale de terrains Les auteurs présentent un système de génération procédurale de terrain combinant trois types d'approches : synthétique, physique et exemple-based (basée sur des exemples). L'objectif est de créer des paysages étendus, composés de différents biomes et peuplés de nombreux éléments.

Le système repose sur un pipeline, structuré en quatre étapes principales. Chaque étape peut être visualisée directement, ce qui améliore la facilité d'utilisation. Chaque étape est placée sur une grille comme présenté dans la figure 6 (b). Ce design offre un bon compromis entre les exigences souvent opposées : performance, réalisme, flexibilité. Voici les différentes étapes de ce pipeline :

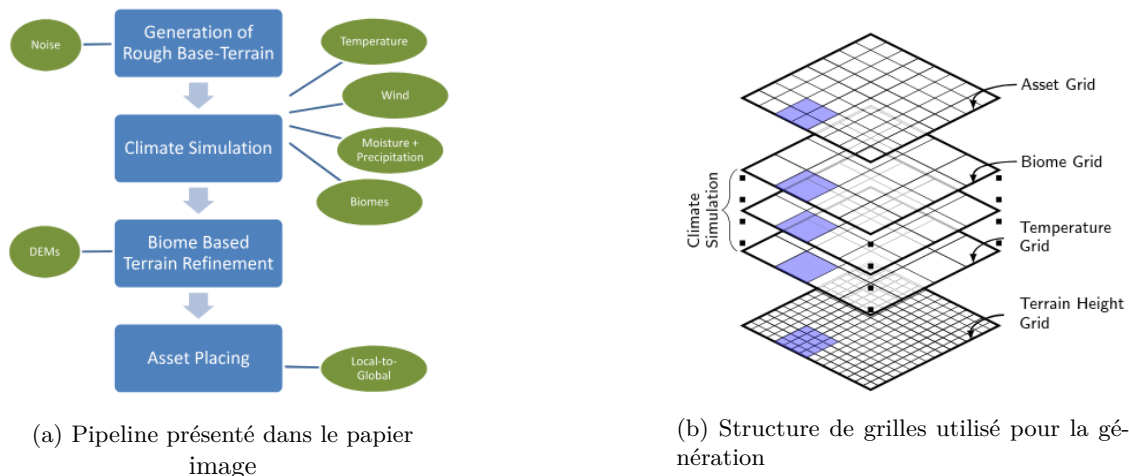


FIGURE 11 – Images tirés du papier

Terrain de Base Le terrain est Généré à l'aide de fonctions de bruit, notamment du bruit simplex [4] multi-octaves. Cela permet une création rapide et flexible d'un terrain général, sans se soucier des détails réalistes dès cette étape. L'utilisateur peut ajuster les paramètres, comme le nombre d'octaves ou le seuil de niveau de la mer.

Simulation climatique Cette étape a pour but de calculer la distribution des biomes la plus réaliste possible. Pour cela aucun bruit n'est utilisé mais une approche basée sur la physique a été choisie elle reste simple pour qu'elle soit relativement rapide contrairement a d'autres méthodes basées sur la simulation physique. Elle se compose de 4 étapes :

- **Température** : Calculée par interpolation bilinéaire ou sinusoïdale, en prenant en compte l'altitude (plus c'est haut, plus c'est froid). Cela permet d'avoir un gradient entre régions chaudes et froides.
- **Vent** : Le vent est simulé en définissant des forces aux quatre coins de la carte. Une approche itérative propage les directions de vent dans un champ vectoriel, en combinant les directions des cellules voisines et en ajoutant de petites perturbations aléatoires pour simuler les micro-variations. La proximité d'un coin renforce la persistance de son influence, produisant ainsi un lissage réaliste ou des annulations aux frontières entre courants principaux.
- **Précipitations** : La répartition des précipitations est calculée de manière itérative, en utilisant les données de vent et de température. Les cellules d'eau servent de sources d'humidité, avec une évaporation dépendant de la température. L'humidité est ensuite transportée par le vent vers les cellules voisines, avec une dispersion partielle vers les cellules adjacentes. Cette méthode permet de de simuler des phénomènes plus avancés comme les ombres pluviométriques ou l'effet de foehn.

- **Placement des biomes** : Calcul de la grille de Biomes en fonction des propriétés calculées, en particulier la température et les précipitations. À chaque paire de valeurs température-précipitation est ainsi associé un identifiant de biome spécifique, selon une table.

Enrichissement du terrain : Pour enrichir le terrain de base, les auteurs utilisent une approche par exemple en intégrant des DEMs (modèles numériques d'élévation) spécifiques à chaque biome, tirés de sources disponibles publiquement. Les DEMs sont une représentation numérique de la hauteur du terrain sur une surface donnée, généralement sous forme d'une grille régulière où chaque cellule contient une valeur d'altitude. Cette méthode offre un réalisme élevé avec peu d'efforts. Pour obtenir des transitions naturelles entre biomes, les frontières sont déformées avec du bruit fractal (simplex noise), générant une grille de biomes à plus haute résolution. Ensuite, une convolution avec un noyau réglable permet de mélanger les DEMs voisins selon la proportion de chaque biome dans la zone. Le résultat est une somme pondérée des DEMs, fusionnée au terrain de base pour produire une carte de hauteur finale réaliste et cohérente.

Placement d'objets : Dans la dernière étape, les biomes sont peuplés par des objets 3D (assets) placés selon un modèle itératif. Ce modèle permet des distributions naturelles et adaptables aux transitions entre biomes. Les objets sont sélectionnés depuis une base d'assets (arbre, rocher, etc.), chacun possédant des propriétés spécifiques (ex. : tolérance à l'ombre, distance minimale, etc.). Le placement se fait via un échantillonnage de type Poisson-disk [9], adapté pour gérer des distributions complexes avec contraintes. Les objets sont traités par catégories (organique/inorganique, petite/grande taille) pour une répartition plus réaliste. On obtient une répartition uniforme et performante d'objets sur le terrain généré.

Conclusion La figure 7 présente des exemples de terrain généré grâce à cette méthode.

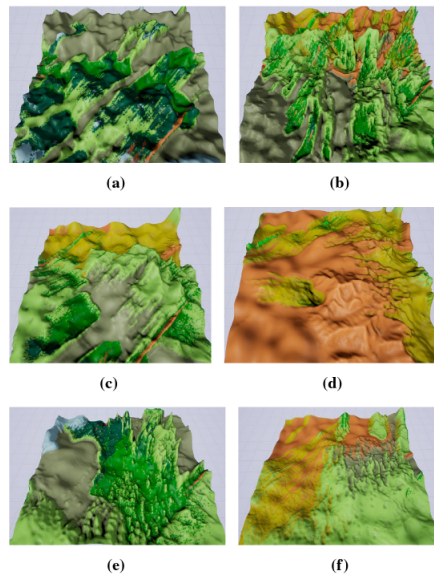


FIGURE 12 – Un ensemble de terrains différents générés avec différents paramètres

Ce papier présente une méthode intéressante pour la génération de terrain et de placement d'objets mais nous n'avons pas décidé de suivre cette méthode pour notre projet, celle-ci est complexe et les résultats présentés ne nous ont pas convaincus. En effet pour une méthode qui met l'accent sur le réalisme nous trouvons les résultats trop aléatoires et peu esthétiques. Nous avons quand même utilisé certaines idées comme la table en fonction de l'élévation et de l'humidité pour les biomes.

3.3 papier 2 UI [\[12\]](#)

3.4 papier 4

4 développement

Le développement de notre projet repose sur l’articulation de deux composants logiciels bien distincts, chacun répondant à des besoins spécifiques et complémentaires. D’une part, nous avons conçu un outil de génération de terrain procédural développé en **C++**. Ce programme est chargé de produire automatiquement des cartes de jeu structurées, de taille paramétrable, intégrant différents *biomes* (zones géographiques thématiques telles que forêt, montagne, désert, etc.) ainsi qu’un positionnement intelligent et configurable des ressources (comme le bois, la pierre ou l’or).

Choix de développement

- Cet outil de génération doit également produire des données dans un format structuré et exploitable par le second programme du projet : un **moteur de jeu** léger, développé en **TypeScript** et reposant sur la bibliothèque **Three.js**, capable de tourner directement dans un navigateur web. Ce moteur assure l’affichage de la carte, la gestion de l’interaction utilisateur (notamment en mode Péon), et l’intégration des mécanismes de jeu à travers une interface en ligne fluide et réactive.
- Le choix de cette architecture repose à la fois sur des considérations techniques (performance, portabilité, modularité) et sur des contraintes pédagogiques liées à notre formation. Le développement de la génération procédurale en C++ nous permet de capitaliser sur notre expérience en programmation c++, tandis que le moteur de jeu web représente un défi plus orienté front-end, mobilisant des compétences en visualisation 3D interactive dans un contexte que nous maîtrisions peu au départ. Il s’agit donc également d’une opportunité d’apprentissage significative, puisque nous avons dû nous approprier rapidement des outils comme Three.js ou encore les principes de développement web modernes, dans un cadre concret et motivant.
- Le choix de TypeScript plutôt que JavaScript pur s’explique par notre volonté de bénéficier d’une meilleure robustesse dans le code, grâce au typage statique et à une meilleure maintenabilité à mesure que le projet grandit. Cette décision s’inscrit dans une logique de professionnalisation du développement et d’anticipation des besoins futurs en matière d’évolutivité.
- La communication entre les deux programmes (générateur C++ et moteur TypeScript) repose sur une série de fichiers intermédiaires, permettant une séparation claire des responsabilités tout en assurant une interopérabilité efficace. Ainsi, notre projet s’organise autour d’un pipeline de production modulaire : génération des données brutes d’un côté, exploitation visuelle et interactive de l’autre.

4.1 Génération de la carte

Pour ce projet nous voulions créer un outil pour créer une carte procéduralement. Pour cela nous avons écrit un programme en C++ qui crée un fichier .OBJ [6] à partir de cartes de bruits.

Cartes de bruits Une carte de bruit, ou "noisemap", dans le contexte de la génération procédurale, est une technique utilisée pour créer des textures, des terrains ou des environnements de manière algorithmique. Les noisemaps dans la génération procédurale sont utilisées pour introduire des variations naturelles et réalistes dans les modèles générés par ordinateur.[1]

La génération d'une noisemap implique l'utilisation de fonctions de bruit, telles que le bruit de Perlin, le bruit de Simplex ou d'autres algorithmes de bruit procédural. Ces fonctions génèrent des valeurs pseudo-aléatoires qui varient de manière continue et douce à travers l'espace. L'utilisation la plus élémentaire de ce genre de cartes est de lier l'élévation du terrain aux valeurs de la carte de bruit.



FIGURE 13 – Visualisation de Carte de bruit

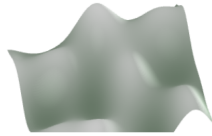
Nous pouvons manipuler l'aléatoire des fonctions qui génèrent ce genre de carte grâce à différents paramètres :

- La fréquence dans le contexte des fonctions de bruit, comme le bruit de Perlin, détermine la granularité ou la finesse des détails dans la carte de bruit générée. Une fréquence élevée signifie que les variations de bruit se produisent plus rapidement, ce qui entraîne des détails plus fins et plus nombreux dans la carte. À l'inverse, une fréquence basse produit des variations plus lentes et plus douces, résultant en des caractéristiques plus larges et moins détaillées.
- Les octaves sont utilisées pour ajouter de la complexité et du réalisme aux cartes de bruit. Chaque octave est une couche supplémentaire de bruit générée à une fréquence différente. En superposant plusieurs octaves, on peut créer des textures plus riches et plus variées. Les octaves sont généralement ajoutées avec des fréquences de plus en plus élevées et des amplitudes de plus en plus faibles.

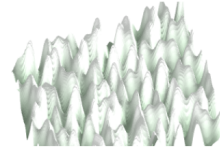
Effet sur la Carte Générée

- *Fréquence Basse* : Une fréquence basse produit des caractéristiques larges et douces. Par exemple, dans la génération de terrains, une fréquence basse peut créer des collines et des vallées larges et douces.
- *Fréquence Élevée* : Une fréquence élevée produit des détails fins et nombreux. Par exemple, dans la génération de terrains, une fréquence élevée peut ajouter des rochers, des crevasses et d'autres détails fins à la surface.
- *Octaves Multiples* : L'utilisation de plusieurs octaves permet de combiner des caractéristiques larges et douces avec des détails fins. Par exemple, une première octave avec une fréquence basse peut créer les grandes formes du terrain, tandis que des octaves supplémentaires avec des fréquences plus élevées ajoutent des détails fins comme des rochers et des textures de surface.

Voici des exemples en images de la différence.



(a) Exemple carte et élévation avec une fréquence basse
image



(b) Exemple carte et élévation avec une haute fréquence

FIGURE 14 – Comparaison de niveau de fréquences

Génération de biomes Sur notre carte nous voulons différents biomes c'est à dire différents environnements répartis le plus naturellement possible. Pour cela en plus de la carte de bruit qui va gérer l'élévation du terrain nous allons générer une autre carte de bruit qui va simuler l'humidité du milieu. En combinant ces deux cartes on peut obtenir des biomes plutôt cohérents. En plus de cela nous pouvons définir une élévation minimale qui va permettre de créer des océans. Enfin nous avons défini une aire de jeu circulaire au milieu de la carte celle ci sera plate mais utilise quand même les noisemap pour gérer les biomes.

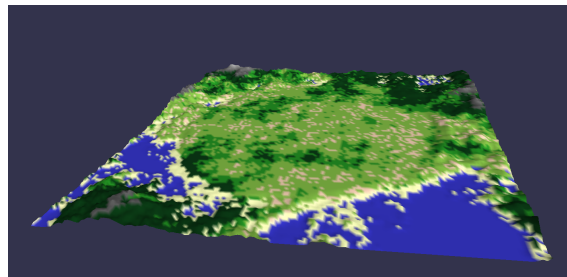
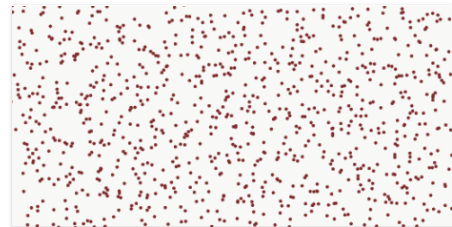


FIGURE 15 – Une carte générée avec notre code

Placer des ressources Un autre défi de la création de la carte est de placer des ressources sur la carte en favorisant les potentielles interaction entre les royaumes, c'est à dire créer des ressources qui seront rares et ne pas les répartir de manière uniforme pour obliger les royaumes à interagir entre eux. Si on prend simplement des positions aléatoires sur la carte par exemple on se retrouve avec un résultat trop uniforme on aimerait avoir une génération avec des trous et des endroits plus concentrés. La solution que nous avons trouvé pour cela ce sont les "Jittered grid"[\[2\]](#) on peut traduire cela par "grille perturbée". Le principe est de générer une grille carrée ou hexagonale et d'appliquer une perturbation aux points de la grille. Voici le résultat.



(a) Grille avant la perturbation



(b) Gille après la perturbation

FIGURE 16 – Visualisation de jittered grid

On voit apparaître des zones avec peu de points et d'autres avec beaucoup de points.

Résultat sur la carte Les triangles violets représentent les positions des ressources.

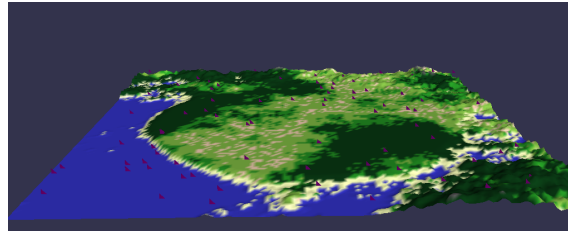


FIGURE 17 – Carte avec position de ressources générés

Il suffira de re-exécuter ce code le même nombre de fois que de ressources. Les emplacements des ressources sont enregistrés dans un fichier csv qui pourra être lu par le moteur de jeu.

Importation sur le moteur Une fois la carte générée grâce à cet outil développé en C++ il faut l'importer sur notre moteur fait maison en TypeScript (Nous en parlons en détail dans la prochaine section). Pour cela nous avons l'option d'enregistrer la carte générée au format OBJ pour la géométrie, on peut récupérer également la texture des couleurs. Pour les ressources l'objectif étant qu'elles puissent être chargés par le serveur et pas directement par le frontend. Pour cela les coordonnées de la ressource sont simplement enregistrés sur un fichier csv qui n'enregistre que ses coordonnées en 2D (x et y) et son type :

```
tree,1.61432,2.03902
tree,1.60876,2.0844
tree,1.61334,2.12896
tree,1.61054,2.16839
tree,1.60979,2.48438
tree,1.61826,2.48242
rock,2.46094,1.875
rock,2.5,-1.44531
rock,2.5,-1.25
rock,2.5,1.32812
gold,-2.46094,-1.48438
gold,-2.46094,-1.21094
gold,-2.30469,1.83594
```

Pour le moment nous allons charger et rendre les ressources directement dans le frontend chose qui sera changé à terme.

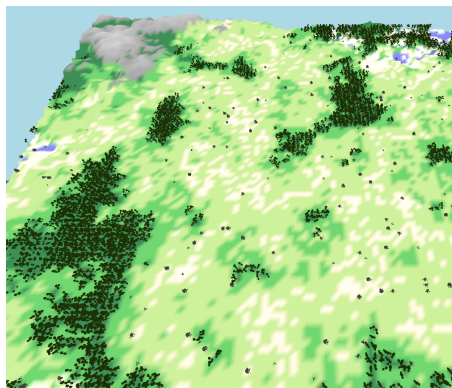


FIGURE 18 – La carte et les ressources sur le moteur maison

4.2 Moteur de jeu

4.2.1 Technologies

L'objectif étant de créer un jeu par navigateur, nous avons dû choisir entre trois approches distinctes :

- **Moteur de jeu existant et export HTML5**
Avec cette méthode, nous utilisons un moteur de jeu (par exemple, Godot ou Unity) pour développer le jeu puis l'exporter en HTML5. Cependant, cela nous donne moins de contrôle sur les performances et les capacités de notre jeu.
- **typescript + ThreeJS**
typescript est une surcouche de javascript permettant un typage fort et une programmation par classe plus solide. *ThreeJS* est une librairie javascript permettant de faire de la 3D sur navigateur (*voir WebGL*) Cette méthode nous oblige à créer beaucoup plus de choses de zéro, mais elle nous offre aussi davantage de contrôle sur le résultat et est la plus intéressante d'un point de vue pédagogique.
- **Angular + ThreeJS**
Angular est un framework en typescript pour la création de sites web. Il propose extension pour fonctionner avec *ThreeJS*. Cette méthode nous facilite la création d'interfaces utilisateur et la gestion de la partie web en général. Cependant, cela entraîne un coût en temps de formation et en performances.

Comme nous sommes des élèves d'Imagine, nous avons décidé de partir sur la deuxième option. C'est celle qui nous permet le mieux de mettre en action et d'améliorer nos compétences. De plus, cela nous donne l'occasion d'apprendre le développement web orienté 3D et applications interactives.

4.2.2 Le design d'un moteur de jeu

Un moteur de jeu est un logiciel intermédiaire (*middleware*) permettant de faciliter la création de jeux vidéos. Il regroupe généralement un ensemble de fonctionnalités (visuel, physique, son, etc). Certains moteurs de jeux sont généraliste (faits pour tous les types de jeux), d'autres sont spécialisés pour certains types de jeux, voire développés pour un unique jeu ou une unique série de jeu.

Un moteur possède presque toujours une boucle de rafraîchissement (*update loop*). Chaque exécution de cette boucle fait avancer dans le temps l'état du jeu, et effectue le rendu graphique du jeu dans ce nouvel état. On préfère habituellement avoir une fréquence de rafraîchissement d'au moins 60 images par secondes pour les applications interactives, alors que 24 IPS ou 29.8 IPS sont souvent suffisants pour des séquences non interactives.

Notre objectif étant de créer un jeu, notre moteur peut être très spécialisé et orienté vers le jeu que nous voulons créer. Cependant, nous allons voir qu'il y a énormément de manières différentes de structurer un jeu vidéo. D'autre part, en développant un moteur, il convient de garder à l'esprit qu'il sera utilisé pour créer des (dans notre cas, un) jeux. Il faut prêter une attention toute particulière à la clarté et à la facilité de compréhension et d'utilisation du code que l'on crée, car la vitesse de développement et la qualité des jeux créés avec le moteur en dépend directement.

La manière classique de structurer un jeu vidéo (ainsi que la majorité des applications 3D interactives) est par l'usage d'un concept d'**entité de jeu**. Classiquement, une entité a une existence dans le monde du jeu, par exemple, une montagne, un personnage, une zone qui détecte d'autres entité, etc.

Les entités sont souvent organisées de manière hiérarchique dans une structure arborescente appelée **Graphe de scène**. Une entité qui est enfante d'une autre hérite généralement de sa transformation 3D : en d'autres termes, les mouvements de l'entité parente sont répercutés sur l'entité enfante (un chevalier qui court entraîne aussi l'épée qu'il tient en main). D'autre part, ces entités sont spécialisées selon leur utilité : une porte, un zombie ou une explosion nécessitent un traitement différent de la part du développeur.

La méthode d'implémentation du graphe de scène et des entités varie beaucoup selon le moteur :

- *Gamebryo*, un moteur de jeu sorti en 1998 et sur lequel sont basés plusieurs moteurs récents (par exemple, celui du jeu *Skyrim*), utilise une hiérarchie d'entités qui peuvent hériter de la transformation et d'autres propriétés de leur parent. Il est par ailleurs possible d'ajouter des comportement aux entités en *LUA* ou en *C++*
- *Godot* utilise un graph de "Noeuds". Chaque noeud est une classe héritée de la classe *Node* lui donnant des comportement spécifiques (par exemple, "corps de simulation physique" ou "maillage 3D").
- *Source Engine* sépare les objets du monde en deux catégories, les objets "Monde", qui sont statiques, et les objets "Entité", qui requiert l'exécution régulière de leur logique (customisable en *LUA*)
- *Unity* utilise un arbre de *Gameobjects*, lesquels contiennent une liste de composants. Les composants spécialisent le comportement de l'objet.
- *Dark Engine*, un autre moteur publié en 1998 et utilisé notamment pour les jeux *System Shock 2* et *Thief*, utilise une architecture entity-component-system, dont nous expliquerons le fonctionnement plus tard.



(a) *Thief*, 1998 utilise une architecture ECS



(b) *Disco Elysium*, 2019, utilise une architecture Entity-Component sur le moteur Unity

Pour faire notre choix parmi toutes ces possibilités (et davantage encore!), nous avons dressé un cahier le cahier des charges de notre jeu : Tout d'abord, nous devons pouvoir afficher et mettre à jour **plusieurs centaines d'entités** sans chute majeure de performances. Nous avons besoin de pouvoir recevoir du serveur un **flux constant** de données et de le propager dans le jeu. Nos entités seraient peu complexes (non-nécessité d'une hiérarchie d'entités), et peu variées dans leur logique (préférence pour une approche *data-driven*).

4.2.3 Entity-component-system

L'architecture Entité-Composant-Système (souvent abrégé ECS) est une architecture permettant de décrire des entités en séparant totalement les données des traitements. Comme son nom l'indique, l'ECS est basé sur trois concepts clés :

- **Les entités** (dans le jargon ECS) représentent exactement ce que l'on a appelé jusqu'ici "entité" : *quelque chose* qui est dans le monde du jeu. Par elles mêmes, elles ne contiennent ni logique ni donnée, mais elles sont chacune associées à un ensemble de composants.
- **Les composants** sont des conteneurs de données typés. Leur type indique la fonction de leurs données. Par exemple, un composant 'Rendu' pourra contenir un maillage 3D et un composant 'Transformation', une matrice de $M_4(\mathbb{R})$. Les composants ne contiennent aucune logique, mais c'est les systèmes qui vont exploiter leurs données.
- **Les systèmes** sont des fonctions qui agissent sur les entités de manière régulière ou ponctuelle. Chaque système n'agit que sur un sous-ensemble des entités qui possèdent les composants nécessaire à l'application du système. Par exemple, un système "Affichage3D" pourrait agir sur toutes les entités ayant au moins les composants "Rendu" et "Transformation".

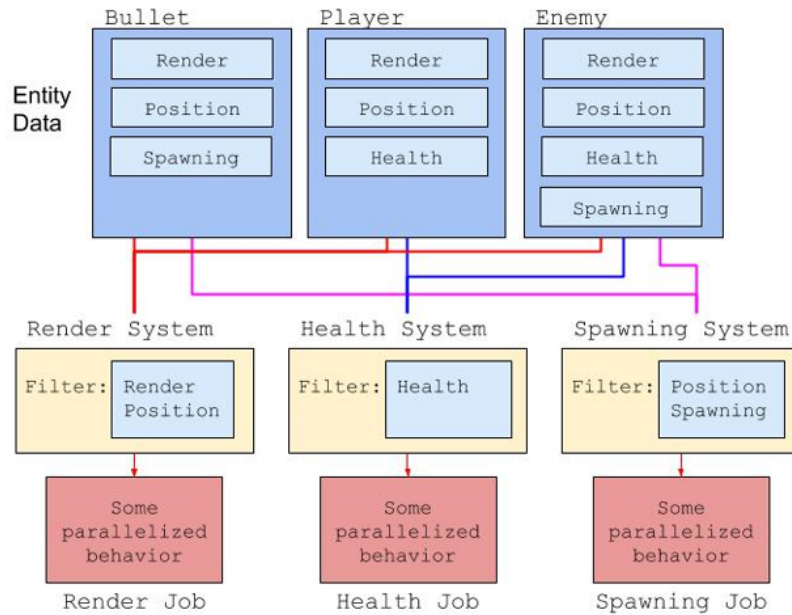


FIGURE 20 – Diagramme produit par Intel illustrant une architecture ECS

Les avantages de l'ECS sont aussi nombreux que ses problèmes. La centralisation des traitements permet de gagner en performance lorsque le nombre d'entités est bien plus grand que le nombre de systèmes (donc que chaque système agit sur beaucoup d'entités en moyenne). En outre, la concentration des données dans les composants permet une approche *data-driven*¹. D'un autre côté, pour des cas d'utilisation avec peu d'entités et beaucoup de variance dans la logique de traitement, l'ECS est peu souhaitable car il est moins intuitif à utiliser que la plupart des alternatives, et les gains de performance sont moindres voir négatifs.

Nous utilisons une architecture ECS car ses forces coïncident avec notre cas d'utilisation : Traitement de beaucoup d'entités avec un nombre de traitement relativement restreint, et spécialisation d'un grand nombre d'entités par une approche orientée-données.

4.2.4 Notre moteur

Pour rappel, nous avons choisi de développer notre propre moteur en *typescript*, en utilisant la librairie de rendu 3D *ThreeJS*.

ThreeJS nous offre une interface haut niveau vers le WebGL. En d'autres termes, on lui délègue le rendu graphique. Cependant, le reste des concepts servant de base au jeu doivent être implémentés par nos soins. On décide de créer un moteur de jeu basique, spécialisé pour le jeu que l'on crée (en opposition à un moteur généraliste comme Godot ou Unreal Engine). Ce moteur est composé de plusieurs modules :

- ECS
- Components
- Systems
- Input
- User Interface
- Game

1. "régie par les données". Paradigme de programmation où les données sont spécifiées en amont des traitements. [XSLT](#) le langage de transformation du XML en est un exemple.

Ces modules fonctionnent ensemble pour fournir une base solide à la création du jeu. Expliquons chacun d'eux en détails.

4.2.5 ECS

Notre implémentation de l'architecture ECS est basée sur [cet article de 2021 écrit par Maxwell Forbes](#). Les entités sont simplement représentées par un entier. La classe *ECS* lie ces identifiants à un ensemble d'objets héritant de *Component*. La classe *ECS* contient aussi l'ensemble des systèmes. Quand un système, une entité ou un composant est ajouté(e) ou enlevé(e) l'ECS, la liste des entités concernées par chaque système est mise à jour. Une fonction permet d'appliquer le traitement de chaque système aux entités concernées. Voici un exemple-jouet d'utilisation de notre système ECS pour faire rebondir des objets sur le sol.

```
// Vitesse et position mis en un seul composant pour l'exemple

class C_VelocityPosition extends Component {
    position: Vector3 = Vector3(0, 0, 0);
    velocity: Vector3 = Vector3(0, 0, 0);
}

class S_Physics extends System {
    override componentsRequired = new Set<Function>([C_VelocityPosition]);

    override onUpdate(entities: Set<Entity>, timeSinceLastUpdate: number): void {
        for (let entity of entities){
            let comps = this.ecs.getComponents(entity);
            let velPos = comps.get(C_Velocity);

            velPos.velocity.add(Vector3(0, -1, 0)); // gravité

            velPos.position.add( // intégration physique
                velPos.velocity.clone().multiplyScalar(deltaTime *
                    this.physicsSpeed)
            );

            // on admet que le sol est un plan en y = -1 et
            // que les entités ont une forme de collision sphérique de rayon 1
            if (velPos.position.y <= -1.0 + 1){
                velPos.position.y = -1.0 + 1;
                velPos.velocity.y = -velPos.velocity.y; // on fait rebondir la
                    // balle.
            }
        }
    }
}
```

Nous définissons une liste de composants et systèmes "Core" : des éléments généraux et fondamentaux au jeu. Comme notre moteur est très lié à notre jeu, cette frontière n'est pas toujours claire, par exemple, le code de synchronisation avec le serveur et le code d'interaction joueur-monde sont tous deux définis dans le jeu. Ils seront explicités dans la partie suivante qui portera sur le jeu en lui même.

4.2.6 Components (Core)

Certains composants sont définis au niveau du moteur, d'autres, plus spécifiques à notre jeu, le sont au niveau du code de jeu. Ces derniers seront abordés plus tard. Voici la liste des composants "Core", ceux du moteur :

- **C_Transform** représente la position, la rotation et la taille d'une entité
- **C_Mesh** représente un maillage 3D, avec potentiellement un *material*²
- **C_InteractionSphere** Une sphère utilisée pour les détections d'interactions avec le monde (voir la partie sur l'interaction)
- **C_Velocity** représente la vitesse d'une entité. Utilisée pour les calculs physiques
- **C_PhysicsObject** marque un objet pour le système de simulation physique
- **C_None** S'utilise pour signifier qu'un système n'agit sur aucune entité

4.2.7 Systems (Core)

A l'instar des composants, plusieurs systèmes sont définis dans le moteur :

- **S_Draw**
Composants nécessaires : {C_Mesh, C_Transform } Ce système s'occupe du rendu visuel des différents objets de la scène 3D
- **S_Physics**
Composants nécessaires : {C_Velocity, C_Transform, C_PhysicsBody }
Ce système effectue une simulation physique très simple sur les entités qu'il traite. Il est fait pour servir à faire des effets visuels sympathiques plutôt que des simulations réalistes. On peut simuler quelques milliers d'objets avec une fréquence d'image satisfaisante
- **S_Replication**
Composants nécessaires : {C_None}
Ce système s'occupe de la synchronisation entre le serveur et le client. Nous en parlerons d'avantage dans la partie sur l'architecture du jeu en lui même.

4.2.8 Input

Dans une application interactive, il est particulièrement fondamental de pouvoir interagir avec l'application (!)

Il y a trois contextes différents dans lesquels on veut récupérer les touches du clavier ou les boutons de la souris :

1. A intervalle régulier (à chaque passage de la boucle d'*update*, par exemple), on vérifie qu'une touche particulière est pressée. C'est comme ça, qu'on détecte les déplacements du personnage tant qu'une touche est pressée
2. De manière événementielle, pour exécuter du code à chaque fois qu'une touche est pressée. C'est utile lorsque le joueur doit faire une action ponctuelle, par exemple, interagir avec les bâtiments de son village.
3. En tant que modificateur d'autres touches, pour les touches de combinaison telles que les touches *control*, *alt* et *shift*.

Javascript ne permet que de récupérer les entrées de manière événementielles grâce à des fonctions de callback. Il nous a donc fallu créer notre propre système pour détecter et garder en mémoire les entrées utilisateur.

Dans cette optique, nous avons créé une classe statique **Input**, fonctionnant en tant que *singleton*. N'importe qui peut interroger le singleton pour quérir des informations sur une entrée en particulier, ou bien enregistrer une fonction de callback par le biais d'un *subscriber pattern*.

Trois classes composent le système de gestion des entrées utilisateurs :

- **Keybind** représente une touche ou combinaison de touches, sur le clavier ou la souris.
- **Input** est la classe statique principale. Elle contient toutes les fonctions permettant d'interagir avec le système d'entrées utilisateurs. On peut y enregistrer des actions (un couple nom-Keybind), y abonner des fonctions de callback, ou bien quérir l'état d'une touche, y compris la position de la souris à l'écran.

2. Un *material* est un objet contenant toutes les informations décrivant une surface spécifique pour le calcul des lumières. C'est les matériaux qui donnent leur apparence aux différents objets 3D.

- **InputEvent** représente un évènement d'appuie de touche ou de clic. Ce sont eux qui sont envoyés en arguments des fonctions de callback. Seules les actions enregistrées par l'utilisateur envoient des évènements.

Comme souvent dans le développement de moteurs, l'un des objectifs principaux est de fournir du code clair et facilement utilisable par les développeurs du jeu en lui-même. Pour démontrer cet aspect, voici un exemple-jouet de l'utilisation de notre système d'entrées utilisateur :

```
function inputCallback(e: InputEvent) {
    if (e.name == "jump")
        console.log("Le personnage saute !");
}

Input.subscribe(inputCallback); // on abonne notre callback au système

// on crée un nouveau keybind.
// C'est là qu'on peut désigner des modificateurs (shift, ctrl, ...)
Input.add_keybind("jump",
    new Keybind("ArrowUp")
)

Input.add_keybind("go_left",
    new Keybind("ArrowLeft", false, true) // ctrl = false, shift = true
)

function uneFonctionQuiTourneAChaqueUpdate(){
    // demander directement l'état d'une touche
    if (Input.is_action_pressed("go_left"))
        console.log("Le personnage marche vers la gauche !");
}
```

4.2.9 User Interface

Tout le long du jeu, le joueur est amené à interagir avec diverses interfaces utilisateur. Que ce soit pour obtenir de simples informations comme les statistiques du joueur ou bien des données plus globales sur le royaume.

Certaines interfaces sont de manière permanente à l'écran (ressources de base, information joueur, ...) qu'il peut vouloir afficher ou non en fonction des situations, alors que d'autres permettent d'agir réellement sur le jeu, comme le choix d'un bâtiment à placer pour le rôle seigneur.

Puisque nous faisons un jeu par navigateur, nous avons assez naturellement choisi d'implémenter ces interfaces en HTML, cependant, avec les diverses ouvertures, fermetures ou altérations dans l'affichage de ces interfaces, il nous fallait un moyen de les modifier dynamiquement à partir du typescript, de manière déclarative. Après une recherche poussée, nous n'avons pas trouvé de bibliothèque d'interfaces déjà existant vraiment adapté à ce que nous voulions. Les solutions existantes étaient soit trop coûteuses en performances (plusieurs secondes de plus pour charger la page), pas assez customisables (par exemple, l'interface des démos de ThreeJS, qui est faite pour des menus rapides pour les développeurs plutôt que pour une interface utilisateur) ou bien nécessitaient un framework additionnel, tel qu'*Angular* ou *ReactJS*.

Nous avons donc décidé de créer notre propre solution d'interface en type script, pour pouvoir en ajouter de nouvelles ou les modifier plus facilement.

Nous avons donc implémenté une classe pour chacun des éléments HTML qu'on utilise dans nos interfaces, tous héritant d'une classe plus globale **UI**. Ainsi, nous pouvons conserver la philosophie de hiérarchie des données de HTML. Chaque **UI** possède une liste d'enfants héritant de **UI**, qui seront aussi ses enfants dans la hiérarchie HTML. Ce polymorphisme permet de définir facilement des interfaces dans d'autres interfaces, nous offrant une modularité permettant, par exemple, de réutiliser des bouts d'interfaces à divers endroits.

Les objets **UI** gardent en mémoire les informations de l'élément qu'ils représentent, comme sa position, son texte pour les éléments de texte, ou sa fonction de callback pour les boutons. Mais les éléments ne sont ajoutés à l'HTML qu'à l'appel de la fonction d'affichage, qui se répercute récursivement pour construire dans le DOM³ l'arborescence définie de manière déclarative en typescript. De plus, pour nous permettre de customiser le style des éléments d'interface chaque objet **UI** référence une classe CSS. Ainsi, il nous suffit de définir au préalable quelques classes de styles dans un fichier CSS, par exemple pour les différentes couleurs de texte.

Certaines informations, comme la position des éléments, et leur mode de positionnement (absolu, relatif par rapport à leur parent, etc) doivent être passées dynamiquement du typescript au CSS. Pour cela, on utilise des variables CSS, une fonctionnalité spéciale permettant de modifier dynamiquement les attributs d'une feuille de style.

Finalement, il n'y a plus qu'à construire notre interface, puis lier nos boutons aux fonctions de callback associées, qui vont généralement ouvrir ou fermer des sous-interfaces, et lier nos textes aux données récupérées du serveur.

4.2.10 Game (Classe)

La classe **Game** permet de représenter le jeu en lui même. C'est elle qui contient l'objet ECS, ainsi que la scène ThreeJS : une arborescence propre à ThreeJS lui permettant de gérer l'affichage des objets 3D. Le jeu contient aussi une référence à la caméra ThreeJS active, en d'autre terme, de quel endroit, dans quelle direction et avec quel champ de vision est rendue la scène. C'est donc la classe **Game** qui s'occupe de l'abstraction entre l'ECS et le système de rendu de ThreeJS.

En outre, c'est ici qu'est créée la boucle de rafraîchissement. C'est elle qui va mettre en mouvement tout le reste du jeu : mettre à jour les systèmes, synchroniser les données, effectuer le rendu graphique du jeu, etc.

Une autre problématique importante dans les jeux est la transmission efficace des données et des événements entre les différentes parties du jeu. Un jeu vidéo possède énormément de parties distinctes qui doivent communiquer. Souvent, un système de signaux est implémenté dans les moteurs pour assurer cette communication : On connecte un événement d'une entité à une fonction sur cette entité ou autre part. Par exemple, si une plaque de pression au sol est activée, on active le piège.

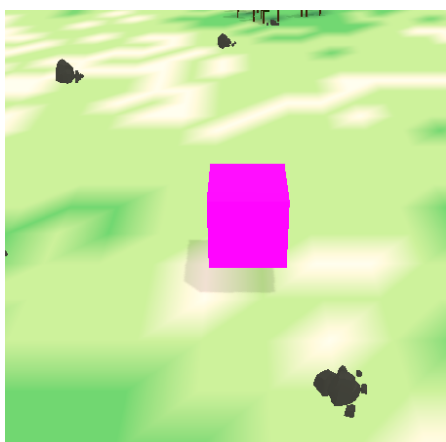
Dans notre cas, nous avons choisi une approche plus rudimentaire mais tout aussi efficace à notre échelle : Un singleton **GameData** contient les informations de manière globale. Chaque partie du jeu peut venir en modifier ou en lire les attributs. On perd l'assurance de la sécurité ("tout le monde peut tout modifier"). Cela a causé plusieurs discussions et débats, cependant, cette solution s'est avérée la plus pratique. La définition de la classe **GameData** sert en quelque sorte de manifeste des données globales à l'échelle du projet, et il est facile avec les outils de développements modernes de tracer ce qui modifie ou lit un champ spécifique du singleton. Les informations comme le mode de caméra (centré sur le personnage ou centré sur la carte) ou bien le rayon de la caméra au curseur de la souris (servant à cliquer dans la scène 3D) sont conservées dans le **GameData**.

3. *Document Object Model*, la version machine de l'arborescence décrite par un fichier HTML

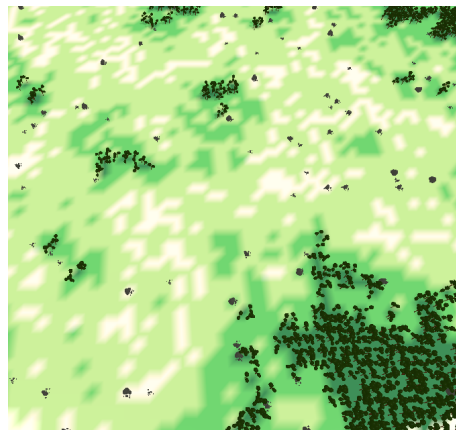
4.3 Implémentation de la caméra

Le contrôle de la caméra dans notre jeu repose sur un système d'interactions combinées entre le clavier et la souris. Afin de centraliser la logique, nous n'utilisons qu'un seul objet caméra, dont le comportement varie légèrement en fonction du mode de jeu actif. Un paramètre dédié permet de définir le *mode de caméra* courant, influençant certaines actions ou limites de mouvement. Nous distinguons actuellement trois modes principaux :

- **Mode Seigneur** : la caméra adopte une vue en 3/4 légèrement inclinée et ne peut pas zoomer au-delà d'une certaine valeur minimale, assurant une vue stratégique du royaume.
- **Mode Carte-Monde** : la caméra est orientée plus verticalement, comme une carte vue de dessus. Le zoom est limité vers le haut afin de conserver une vision globale et simplifiée du monde. Ce mode est conçu pour la navigation macro entre les différents royaumes ou zones.
- **Mode Péon** similaire au mode Seigneur dans sa présentation, il ajoute cependant un suivi automatique du personnage contrôlé par le joueur. Contrairement au mode Seigneur, ce mode repose sur l'existence d'un avatar jouable.



(a) Camera mode péon (le cube représente un personnage)



(b) Camera mode Carte-Monde

FIGURE 21

Le déplacement latéral de la caméra est déclenché par la position de la souris : lorsqu'elle s'approche des bords de l'écran, la caméra se déplace dans la direction correspondante. La vitesse de déplacement est proportionnelle à la proximité du curseur avec le bord, rendant le contrôle plus intuitif.

La recentration de la caméra est possible à tout moment via les touches **Espace** ou **C**. La cible de recentrage dépend du mode : il s'agit de la position du personnage joueur en mode Péon, ou d'un point arbitraire prédéfini en mode seigneur.

Le zoom est géré via la molette de la souris, modifiant uniquement l'altitude de la caméra. Cette variation de hauteur déclenche également un **changement automatique de mode** lorsque certaines valeurs seuils sont franchies. Par exemple, un dézoom important bascule la caméra en mode Carte-Monde, tandis qu'un zoom vers l'avant permet de revenir au mode Seigneur ou Péon. Ce système de transition continue renforce la fluidité de la navigation et justifie l'utilisation d'une unique caméra partagée entre les modes.

Afin de garantir une transition naturelle entre les modes, nous avons mis en place une **zone de seuil élargie** : la caméra entre en mode Carte-Monde uniquement après avoir dépassé une limite haute, et ne revient au mode précédent qu'après avoir atteint une limite basse, évitant ainsi les basculements trop fréquents ou brusques.

Les paramètres relatifs au comportement de la caméra, comme la vitesse de déplacement, les seuils de zoom, ou encore les marges de détection des bords d'écran, sont définis sous forme de constantes, ce qui permet de les ajuster facilement sans avoir à modifier la logique du code.

Enfin, un problème rencontré concernait le comportement de la souris en dehors du canvas du jeu. Lorsque l'utilisateur navigue sur d'autres onglets ou déplace le curseur hors de la fenêtre, la caméra pouvait continuer à se déplacer involontairement. Nous avons donc ajouté une détection explicite de sortie de l'écran, désactivant temporairement le contrôle de la souris jusqu'à son retour dans l'aire de jeu.

En conclusion, nous avons conçu une caméra simple mais polyvalente, capable de s'adapter aux différents contextes de jeu tout en assurant une expérience utilisateur fluide et agréable. Ce système unifié facilite aussi les transitions visuelles et renforce la cohérence entre les différentes phases de gameplay.

4.4 Assets

En complément des fichiers générés par notre outil de génération procédurale de carte, il nous était indispensable de disposer d'**objets 3D** pour représenter visuellement les différentes entités du jeu : ressources naturelles (arbres, rochers, gisements), bâtiments (maisons, ateliers, casernes, etc.), personnages (Péons), et autres éléments décoratifs ou interactifs. Ces assets constituent la base visuelle qui donne vie à notre monde virtuel et permettent de proposer une expérience immersive et cohérente aux joueurs.

Compte tenu du temps limité imparti au projet et de nos ressources humaines, nous avons fait le choix stratégique de **ne pas modéliser nous-mêmes ces objets 3D**. En effet, la création d'assets personnalisés demande non seulement des compétences artistiques spécifiques (modélisation, texturage, rigging), mais également un temps de production non négligeable que nous avons préféré consacrer au développement technique et fonctionnel du jeu.

Nous nous sommes donc tournés vers des bibliothèques en ligne proposant des modèles 3D libres de droits[3]. Après quelques recherches, nous avons sélectionné une collection d'objets au style **low poly**, à la fois léger à afficher et esthétiquement cohérent avec notre direction artistique. Le style low poly a en outre l'avantage d'être performant pour le rendu en temps réel dans un navigateur, tout en gardant une identité visuelle marquée et facilement identifiable.



FIGURE 22 – Exemples d'assets low poly utilisés

Ces assets ont été intégrés dans le moteur de jeu.

Limites et perspectives Même si cette solution nous a permis de gagner un temps considérable, elle présente certaines limites. Tout d'abord, le style visuel du jeu dépend actuellement de ressources externes que nous ne maîtrisons pas totalement. Ensuite, l'uniformité graphique peut parfois être compromise si les assets proviennent de sources différentes. Enfin, l'utilisation de modèles génériques réduit la capacité à représenter

4.5 communication avec le backend

5 Conclusion

Références

- [1] URL : <https://www.redblobgames.com/maps/terrain-from-noise/> (cf. p. 17).
- [2] URL : <https://www.redblobgames.com/x/1830-jittered-grid/> (cf. p. 18).
- [3] *assetPack*. URL : <https://quaternius.com/packs/ultimatefantasyrts.html> (cf. p. 29).
- [4] *bruit simplex*. URL : https://en.wikipedia.org/wiki/Simplex_noise (cf. p. 12).
- [5] Roland FISCHER et al. “AutoBiomes : procedural generation of multi-biome landscapes”. In : *online* (2020). URL : <https://link.springer.com/article/10.1007/s00371-020-01920-7> (cf. p. 12).
- [6] *format obj*. URL : https://en.wikipedia.org/wiki/Wavefront_.obj_file (cf. p. 17).
- [7] Eric GALIN et al. “A Review of Digital Terrain Modeling”. In : *lirmm* (2019). URL : https://seafire.lirmm.fr/d/feab97747cc0474eb0fc/files/?p=%2F16_A_Review_of_Digital_Terrain_Modeling.pdf (cf. p. 8).
- [8] MANDELBROT. “fractional brownian motions, fractional noises and applications.” In : *SIAM Review* (1968) (cf. p. 9).
- [9] *poisson-disk-sampling*. URL : <https://sighack.com/post/poisson-disk-sampling-bridsons-algorithm> (cf. p. 13).
- [10] *r/place*. 2023. URL : <https://www.reddit.com/r/place/> (cf. p. 2).
- [11] *site figma*. URL : <https://www.figma.com> (cf. p. 5).
- [12] Khairul YUSRI et Bin ZAMRI. “The Effects of 10 User Interface (UI) Elements on Game Design Process”. In : *researchgate* (2022). URL : https://www.researchgate.net/publication/354527166_The_Effect_Of_10_User_Interface_UI_Elements_On_Game_Design_Process (cf. p. 14).